Karadeniz Teknik Üniversitesi

Fen Fakültesi – Bilgisayar Bilimleri Bölümü

STL Kapsayıcılara Giriş

Bölüm 1 – Hafta 1

Yazılım Geliştirme II (BILB2000)

Bahar Dönemi – 1. Hafta

Bu Haftanın Planı

1. Ünite 1: STL'e Genel Bakış
2. Ünite 2: Sıralı Kapsayıcılar
3. Ünite 3: İlişkisel Kapsayıcılar
4. Ünite 4: Sırasız Kapsayıcılar ve Uyarlayıcılar
5. Özet

Ünite 1: STL'e Genel Bakış

STL Nedir?

Tanım

Standard Template Library (STL), C++ standart kütüphanesinin bir parçası olan, **jenerik programlama** ilkeleri üzerine inşa edilmiş veri yapıları ve algoritmalar koleksiyonudur.

Alexander Stepanov tarafından 1994'te tasarlandı, C++98 ile standart oldu.

Neden Önemli?

Verimlilik + Güvenilirlik + Üretkenlik

STL Olmadan vs STL ile

STL Olmadan:

```
struct OgrenciListesi {
    Ogrenci* veriler;
    int kapasite, boyut;

    void ekle(const Ogrenci& o) {
        if (boyut >= kapasite) {
            kapasite *= 2;
            Ogrenci* yeni =
              new Ogrenci[kapasite];
            // ... kopyala, sil
        }
        veriler[boyut++] = o;
    }
    ~OgrenciListesi() {
        delete[] veriler;
    }
};
```

STL ile:

```
#include <vector>

std::vector<Ogrenci> ogrenciler;
ogrenciler.push_back(ogr);
// Hepsi bu kadar!
```

Avantaj

Bellek yönetimi, kapasite kontrolü, kopyalama – hepsi otomatik!

Jenerik Programlama ve Template

Jenerik Programlama

Algoritmaların ve veri yapılarının **tür bağımsız** olarak yazılmasını hedefleyen paradigma.

C++'ta `template` mekanizması ile gerçekleştirilir:

- Bir fonksiyon/sınıf yaz, her türle çalışsın
- Derleyici `T` yerine gereken türü koyar
- **Şablon somutlaştırma** (template instantiation)
- Geçen dönem yazdığın `Depo<T>` bir şablon sınıfıydı!

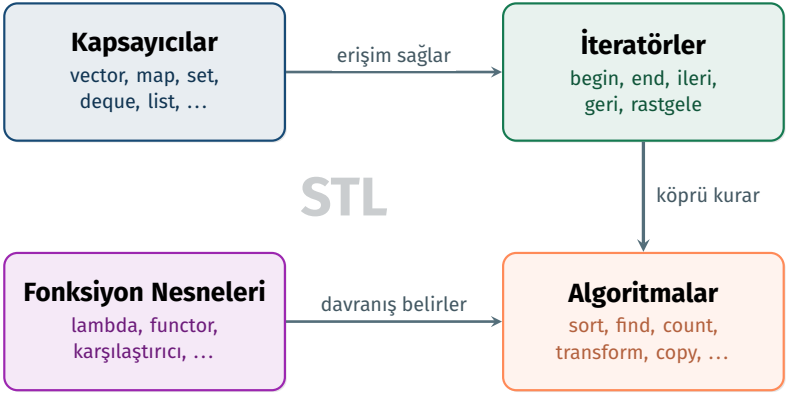
Template Örneği

```
// Her tur icin ayri fonksiyon yerine:
template <typename T>
T maksimum(T a, T b) {
    return (a > b) ? a : b;
}

// Kullanim:
int s1 = maksimum(3, 7);            // T = int
double s2 = maksimum(3.14, 2.71); // T = double
```

Derleme Zamanı Polimorfizmi

Template → tür derleme zamanında belirlenir → çalışma zamanı maliyeti yok!

STL'in Dört Ana Bileşeni



Bileşenlerin Birlikte Çalışması

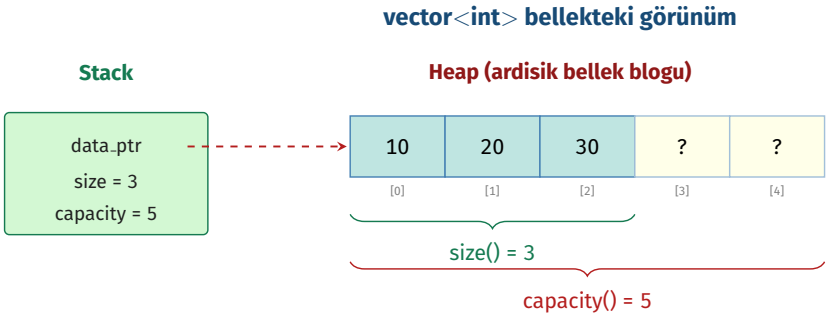
```
#include <vector>       // Kapsayici
#include <algorithm>    // Algoritma

std::vector<int> notlar = {85, 92, 78, 95, 88, 72, 91};

// Kapsayici + Iterator + Algoritma + Fonksiyon nesnesi
int basarili = std::count_if(
    notlar.begin(),    // Iterator: baslangic
    notlar.end(),      // Iterator: bitis
    [](int n) { return n >= 90; }  // Lambda
);
// basarili = 3
```

Tasarım Felsefesi

M kapsayıcı + N algoritma = sadece $M + N$ bileşen (iteratörler sayesinde $M \times N$ değil!)

Ünite 2: Sıralı Kapsayıcılar

vector: Bellek Modeli



Dolu hücreler: kullanılan elemanlar Boş hücreler: tahsis edilmiş ama kullanılmamış alan

- `size()`: Mevcut eleman sayısı
- `capacity()`: Tahsis edilen toplam alan
- Ardışık bellek → CPU önbelleği ile mükemmel uyum

vector: Büyüme Stratejisi

Kapasite dolduğunda **yeniden tahsis** (reallocation):

1. Daha büyük yeni bellek bloğu tahsis et
2. Tüm elemanları yeni bloğa kopyala/taşı
3. Eski bloğu serbest bırak

Kapasite artışı:
$1 \to 2 \to 4 \to 8 \to 16 \to 32 \ldots$

Dikkat

Yeniden tahsis$= O(n)$
Tüm iteratörler geçersiz olur!

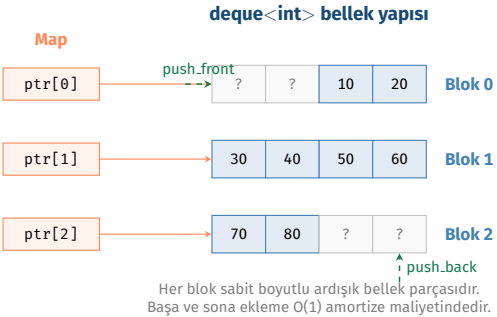
Çözüm

`reserve(n)` ile önceden yer ayır $\to$ gereksiz tahsisleri önle

vector: Temel İşlemler

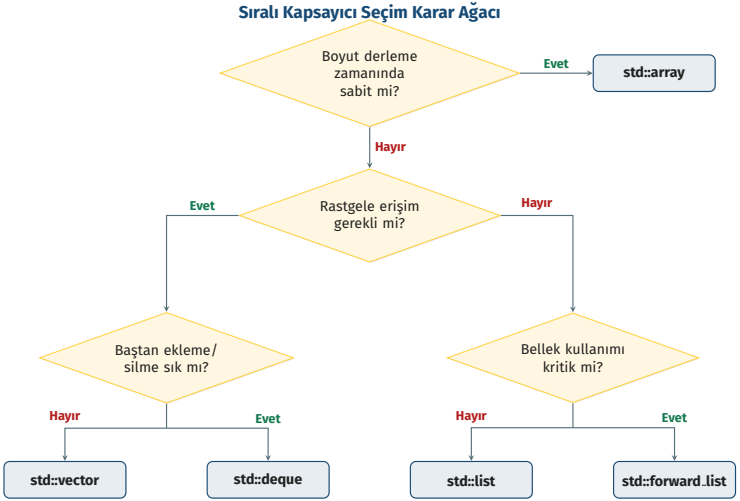
```
std::vector<std::string> ogrenciler;
ogrenciler.reserve(1000);  // Onceden yer ayir

// Ekleme
ogrenciler.push_back("Ali");       // Kopyala + ekle
ogrenciler.emplace_back("Veli");   // Yerinde insa et

// Guvenli erisim
try {
    auto isim = ogrenciler.at(5);  // Sinir kontrolu yapar
} catch (const std::out_of_range& e) {
    std::cerr << "Hata: " << e.what() << '\n';
}

// Tehlikeli erisim
auto x = ogrenciler[999]; // Sinir kontrolu YOK!
```

Diğer Sıralı Kapsayıcılar



Kapsayıcı	Özellik	Kullanım Alanı
`deque`	Baştan + sondan $O(1)$ ekleme	Kuyruk, geri alma
`list`	Herhangi konumda $O(1)$ ekleme/silme	Sık ekleme/silme
`forward_list`	Tek yönlü, az bellek	Bellek kritik
`array`	Sabit boyut, stack üzerinde	Boyut sabit

Sıralı Kapsayıcı Seçim Kriterleri



Altın Kural

Şüphe duyduğunda `vector` ile başla!

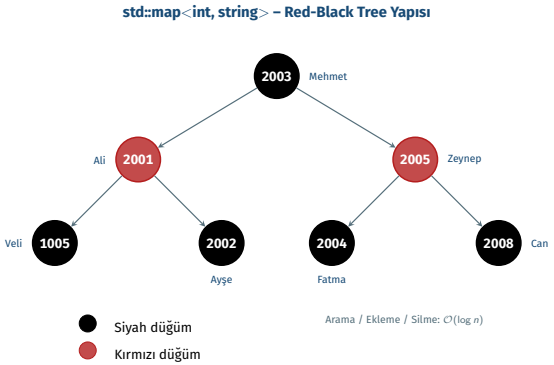
Ünite 3: İlişkisel Kapsayıcılar

std::map – Anahtar-Değer Eşlemesi

Tanım

`std::map<Key, Value>`: Her anahtarın tam olarak bir değerle eşleştiği, **sıralı** ilişkisel kapsayıcı. Dahili yapı: **Kırmızı-siyah ağaç** (red-black tree)

- Arama, ekleme, silme: $O(\log n)$
- Anahtarlar her zaman sıralı
- OBYS: öğrenci numarası → öğrenci nesnesi



map: Ekleme ve Erişim

```
std::map<int, std::string> ogrenciler;

// 3 farkli ekleme yontemi
ogrenciler.insert({2001, "Ali Yilmaz"});
ogrenciler[2002] = "Ayse Kaya";          // [] ile
ogrenciler.emplace(2003, "Mehmet D."); // emplace

// Arama yontemleri
auto it = ogrenciler.find(2001);         // iterator
std::string s = ogrenciler.at(2002);     // exception
if (ogrenciler.contains(2003)) { ... } // C++20

// C++17 structured bindings ile dolasma
for (const auto& [no, isim] : ogrenciler) {
    std::cout << no << ": " << isim << "\n";
}
```

Dikkat: operator[] Tuzağı

Var olmayan anahtara [] ile erişirsen, **otomatik ekler**!

OBYS: vector'den map'e Dönüşüm

Önce (vector):

```
// O(n) arama - yavas!
shared_ptr<T> bul(int id) {
    for (auto& v : veriler) {
        if (v->getId() == id)
            return v;
    }
    return nullptr;
}
```

Sonra (map):

```
// O(log n) arama - hizli!
shared_ptr<T> bul(int id) {
    auto it = veriler.find(id);
    if (it != veriler.end())
        return it->second;
    return nullptr;
}
```

Performans Kazancı

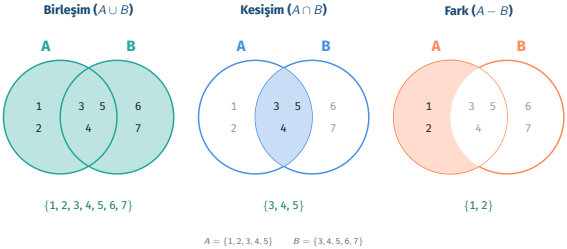
100.000 kayıtta: 50.000 karşılaştırma → 17 karşılaştırma!

std::set – Benzersiz ve Sıralı Kümeler

Tanım

`std::set<T>`: Benzersiz elemanları otomatik sıralı tutan kapsayıcı.

- Aynı elemandan **sadece bir tane**
- Kırmızı-siyah ağaç yapısı (map ile aynı)
- Küme işlemleri: birleşim, kesişim, fark



OBYS

Ders kodları: benzersizlik + sıralı listeleme

set vs map

Özellik	set	map
Sakladığı veri	Yalnızca anahtar	Anahtar-değer çifti
[] operatörü	Yok	Var
Tipik kullanım	Üyelik kontrolü	Eşleme/sözlük
OBYS örneği	Ders kodları	No → öğrenci

multi- varyantları:

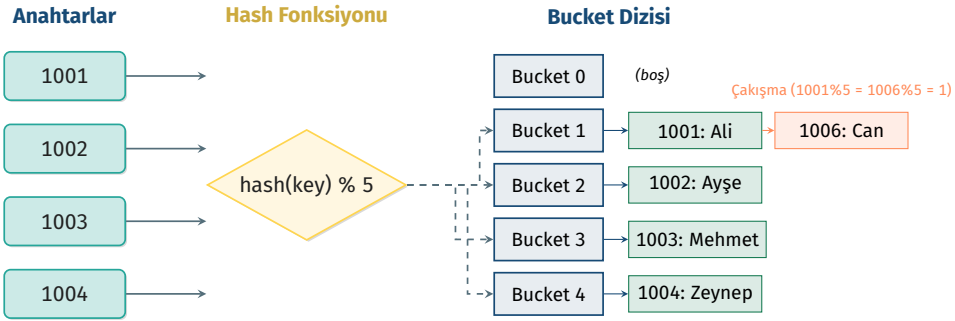
- `multimap`: Aynı anahtardan birden fazla
- `multiset`: Tekrarlı elemanlara izin

Örnek:

- `multimap`: öğrenci → dersler
- `multiset`: sınav notları

Ünite 4: Sırasız Kapsayıcılar ve Uyarlayıcılar

Hash Tablosu Nasıl Çalışır?



1. **Hash fonksiyonu**: Anahtar → tam sayı
2. **Bucket**: `hash(key) % bucket_count` ile konum belirlenir
3. **Çakışma**: Aynı bucket'a düşen elemanlar zincirlenir

unordered_map: $O(1)$ Ortalama Arama

```
#include <unordered_map>

std::unordered_map<int, std::string> ogrenciler;
ogrenciler[1001] = "Ali Yilmaz";
ogrenciler[1002] = "Ayse Kaya";
ogrenciler.insert({1003, "Mehmet Demir"});

// O(1) ortalama arama
if (auto it = ogrenciler.find(1002);
    it != ogrenciler.end()) {
    std::cout << "Bulundu: " << it->second << '\n';
}

// Dolasma sirasi BELIRSIZ!
for (const auto& [no, isim] : ogrenciler) {
    std::cout << no << ": " << isim << '\n';
}
```

Sıralı vs Sırasız Karşılaştırma

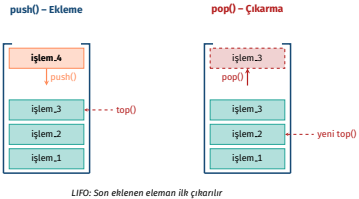
İşlem	map/set	unordered	Not
Arama	$O(\log n)$	$O(1)$ ort.	Hash çakışmasına bağlı
Ekleme	$O(\log n)$	$O(1)$ ort.	Rehashing hariç
Silme	$O(\log n)$	$O(1)$ ort.	
Sıralı gez.	$O(n)$	$O(n)$	Sırasızda sıra yok
Bellek	Az	Fazla	Ödünleşim

Seçim Kuralları

- Sıralı erişim gerekli → `map/set`
- Sadece hızlı arama → `unordered_map/set`

Kapsayıcı Uyarlayıcıları

Var olan bir kapsayıcıyı **kısıtlayarak** özel davranış sunarlar:



stack (LIFO)

Son giren ilk çıkar

Geri alma mekanizması

queue (FIFO)

İlk giren ilk çıkar

İşlem sırası yönetimi

Ders kayıt kuyruğu

priority_queue

Öncelik sırasına göre

Max-heap yapısı

Acil işlem kuyruğu

Uyarlayıcılar kendi başına veri saklamaz

`stack` → `deque` üzerine, `queue` → `deque` üzerine, `priority_queue` → `vector` üzerine

stack ile Geri Alma Mekanizması

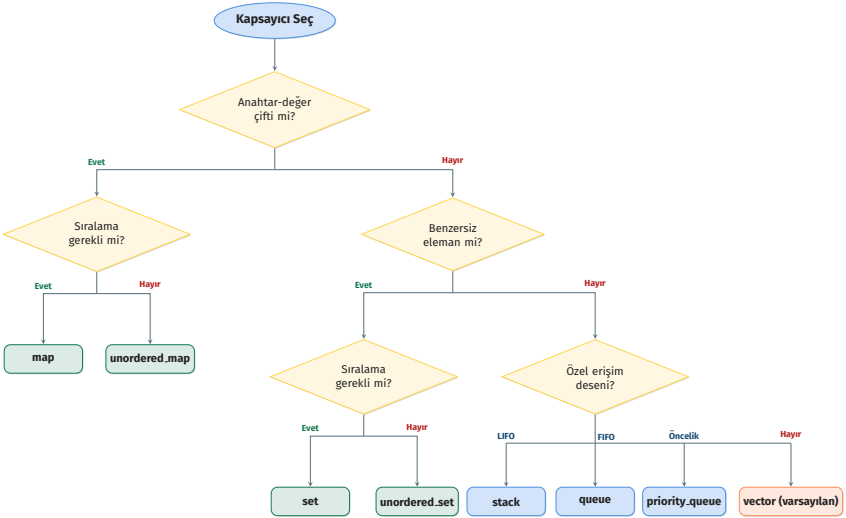
```
#include <stack>
#include <string>

std::stack<std::string> islem_gecmisi;

// Islemleri kaydet
islem_gecmisi.push("Ogrenci 1001 eklendi");
islem_gecmisi.push("Ogrenci 1002 eklendi");
islem_gecmisi.push("Ogrenci 1001 notu guncellendi: 85");

// Geri alma
std::cout << "Geri aliniyor: "
          << islem_gecmisi.top() << '\n';
islem_gecmisi.pop();  // Degeri DONDURMEZ!
```

Önemli

`pop()` değer döndürmez! Önce `top()`, sonra `pop()` çağır.

Özet

Kapsayıcı Seçim Karar Ağacı



Genel Performans Tablosu

Kapsayıcı	Erişim	Arama	Sona Ekle	Ortaya Ekle	Silme	Sıralı
`vector`	$O(1)$	$O(n)$	$O(1)$*	$O(n)$	$O(n)$	Hayır
`deque`	$O(1)$	$O(n)$	$O(1)$*	$O(n)$	$O(n)$	Hayır
`list`	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(1)$	Hayır
`map`	$O(\log n)$	$O(\log n)$	$O(\log n)$	–	$O(\log n)$	Evet
`set`	–	$O(\log n)$	$O(\log n)$	–	$O(\log n)$	Evet
`unordered_map`	$O(1)$	$O(1)$	$O(1)$*	–	$O(1)$	Hayır
`unordered_set`	–	$O(1)$	$O(1)$*	–	$O(1)$	Hayır

* Amortize karmaşıklık

OBYS'de Kapsayıcı Seçimleri

OBYS Bileşeni	Kapsayıcı	Neden?
Öğrenci deposu	`unordered_map`	Hızlı no ile arama
Ders listesi	`map`	Koda göre sıralı
Kayıtlı dersler	`unordered_set`	Hızlı üyelik testi
Not listesi	`vector`	Sıralı erişim
Geri alma	`stack`	LIFO
Kayıt istekleri	`queue`	FIFO
Acil işlemler	`priority_queue`	Öncelik sırası

Bu Hafta Öğrendiklerimiz

1. **STL**: Jenerik programlama üzerine kurulu veri yapıları + algoritmalar
2. **Template**: Tür bağımsız kod yazma mekanizması
3. **vector**: Ardışık bellek, $O(1)$ erişim, amortize $O(1)$ ekleme
4. **deque, list, array**: Farklı senaryolar için alternatifler
5. **map/set**: Kırmızı-siyah ağaç, $O(\log n)$, sıralı
6. **unordered_map/set**: Hash tablosu, $O(1)$ ortalama
7. **stack, queue, priority_queue**: Uyarlayıcılar
8. **OBYS dönüşümü**: `vector` $\rightarrow$ `map` ile hızlı arama

Gelecek Hafta: İteratörler ve STL Algoritmaları

- İteratör kategorileri ve adaptörleri
- Temel algoritmalar: `sort`, `find`, `transform`
- İleri algoritmalar: `accumulate`, `partition`
- **Lambda ifadeleri** ve `std::function`
- OBYS: Filtreleme, sıralama ve arama yetenekleri

Hazırlık

Kitabın Bölüm 2'yi okuyun ve `Depo<T>` sınıfını `map` ile yeniden yazın.